

Zadatak 2

Igra Connect4

Implementacija

Paralelna implementacija algoritma je ostvarena koristeći jedan glavni proces i proizvoljan broj radnika.

Radnici na početku svakog poteza traže zadatak i čekaju na odgovor. Taj odgovor može označavati kraj računanja trenutnog poteza, kraj igre ili može sadržavati zadatak. Ako je poruka zadatak onda ga odrađuju i šalju nazad rezultat, nakon čega čekaju novi zadatak. Kod 1 prikazuje programsko ostvarenje opisanog.

```
bool finish = false;
while (true) {
    if (finish) break;

    MPI_Send(nullptr, 0, MPI_BYTE, 0, TAG_SPECIAL, MPI_COMM_WORLD); // Ask for the first task
    while (true) {
        MPI_Probe(0, MPI_ANY_TAG, MPI_COMM_WORLD, &status); // Wait for a task
        if (status.MPI_TAG == TAG_SPECIAL) { // No more tasks
            MPI_Recv(nullptr, 0, MPI_BYTE, 0, TAG_SPECIAL, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            break;
        } else if (status.MPI_TAG == TAG_FINISH) { // The game is finished
            MPI_Recv(nullptr, 0, MPI_BYTE, 0, TAG_FINISH, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            finish = true;
            break;
        }

        // Obtain task resources:
        MPI_Recv(spots.data(), config.width * config.height, MPI_CHAR, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
        int task_id = status.MPI_TAG;
        MPI_Recv(heights.data(), config.width * sizeof(std::size_t), MPI_BYTE, 0, MPI_ANY_TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        board.set_board(spots, heights);

        double result = comp.move_recursive(board, config.solo_depth + 1); // Perform task
        MPI_Send(&result, 1, MPI_DOUBLE, 0, task_id, MPI_COMM_WORLD); // Send result
    }
}
```

Kod 1: Radna petlja radnika

Glavni proces na početku poteza stvara dvije stablaste strukture kao što prikazuje Kod 2. Jedna struktura služi za pohranjivanje dobivenih rezultata i njihovo agregiranje prema roditeljskom čvoru kada se sakupe rezultati svih čvorova djece, a broj sakupljenih rezultata prati se drugom strukturom.

```
std::vector<std::vector<double>> buffer_results;  
std::vector<std::vector<std::size_t>> buffer_counts;  
std::size_t size = 1;  
for (std::size_t i = 0 ; i <= solo_depth + 1 ; i++) {  
    buffer_results.push_back(std::vector<double>(size, 0.0));  
    if (i != solo_depth + 1) buffer_counts.push_back(std::vector<std::size_t>(size, 0));  
    size *= width;  
}
```

Kod 2: Stvaranje stablastih struktura za pohranu rezultata

Glavni proces zatim rekurzivnom dubinskom pretragom pretražuje prostor stanja te kada dođe do zadane dubine za prosljeđivanje zadatka radnicima čeka da mu se radnik javi te od njega preuzima rješenje prethodno odrađenog zadatka i šalje novi zadatak što prikazuje Kod 3.

```
if (depth == solo_depth) {  
    MPI_Probe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);  
    if (status.MPI_TAG == TAG_SPECIAL) { // If asking for the first task  
        MPI_Recv(nullptr, 0, MPI_BYTE, status.MPI_SOURCE, TAG_SPECIAL, MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
    } else { // We're getting a result  
        MPI_Recv(&result, 1, MPI_DOUBLE, status.MPI_SOURCE, status.MPI_TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
    }  
  
    // Send new task:  
    MPI_Send(board.get_spots_data(), board.get_width() * board.get_height(), MPI_CHAR, status.MPI_SOURCE, task_idx_start + i, MPI_COMM_WORLD);  
    MPI_Send(board.get_heights_data(), board.get_width() * sizeof(std::size_t), MPI_BYTE, status.MPI_SOURCE, task_idx_start + i, MPI_COMM_WORLD);  
  
    if (status.MPI_TAG != TAG_SPECIAL) { // Store result  
        save_result_parallel(status.MPI_TAG, result, width, buffer_results, buffer_counts);  
    }  
} else {  
    move_recursive_parallel(board, (task_idx_start + i) * width, buffer_results, buffer_counts, depth + 1);  
}
```

Kod 3: Komunikacija glavnog procesa u rekurzivnoj funkciji

Ispis zadnjih poteza igre u kojoj računalno pobjeđuje prikazan je na Slika 1.

```

. . X . . . .
. . 0 0 . . .
. X 0 0 0 . .
. 0 0 X X . X
. X X 0 X . 0
. X 0 X X . 0
0 1 2 3 4 5 6
Your move: 5
. . X . . . .
. . 0 0 . . .
. X 0 0 0 . .
. 0 0 X X . X
. X X 0 X . 0
. X 0 X X X 0
0 1 2 3 4 5 6
. . X . . . .
. . 0 0 0 . .
. X 0 0 0 . .
. 0 0 X X . X
. X X 0 X . 0
. X 0 X X X 0
0 1 2 3 4 5 6
Your move: 1
. . X . . . .
. X 0 0 0 . .
. X 0 0 0 . .
. 0 0 X X . X
. X X 0 X . 0
. X 0 X X X 0
0 1 2 3 4 5 6
. . X . 0 . .
. X 0 0 0 . .
. X 0 0 0 . .
. 0 0 X X . X
. X X 0 X . 0
. X 0 X X X 0
0 1 2 3 4 5 6
Computer wins!

```

*Slika 1: Ispis
zadnjih nekoliko
poteza igre u
kojoj pobjeđuje
računalo*

Kvantitativna analiza

Konstante mjerenja:

- širina ploče = 7
- visina ploče = 6
- dubina pretraživanja = 9

Trajanje sekvencijalnog algoritma -> **10644ms**

Mjerenja su odrađena na prvom potezu računala, nakon što igrač odigra potez 3. Rezultati mjerenja vidljivi su na kasnijim stranicama.

Kada promatramo dubinu aglomeracije 1, gdje imamo samo 7 zadataka, najprije dobivamo velika povećanja ubrzanja povećanjem broja procesora kao što bih i očekivali jer svaki jer je najveća broj zadataka kojeg dobije jedan procesor sve manji. Pri prelasku s 5 na 6 procesora je povećanje ubrzanja nešto manje jer će vjerojatno se najveći broj zadataka jednog procesora vjerojatno neće promijeniti (bit će 2), ali ipak postoji malo ubrzanje jer procesor koji prije završi može ići izvršavati preostale zadatke. Učinkovitost nam upravo pokazuje ovo smanjenje dobitaka pri dodavanju dodatnog procesora. Očekivali bi dodatni skok na 8 procesora kada bi svaki dobio jedan zadatak.

Povećanjem dubine aglomeracije na 2 i 3, i time povećanjem broja zadataka postićemo veća ubrzanja za brojeve procesora veće od 2 zbog sitnozrnatosti te nećemo imati beskorisne procesore kao u slučaju sa 7 zadataka koji završe ranije i čekaju ostale da završe. Smanjenjem zrnatosti se smanjuje razlika između potrebnih vremena izvođenja pojedinih zadataka pa možemo postići bolju raspodjelu. Ova razlika u potrebnom vremenu izvođenja pojedinih zadataka bi bila očitija u kasnijim koracima igre kada postoje značajne razlike u dubinama podstabla.

Pri prelasku s dubine aglomeracije 3 na 4 više nemamo poboljšanje ubrzanja i učinkovitosti iz dva glavna razloga. Jedan je taj što imamo znatni porast broja skupih operacija komunikacije, a razlika između vremena izvođenja pojedinih zadataka ne opada značajno i više ne dobivamo značajne prednosti od smanjenja zrnatosti. Drugi razlog je povećanje količine posla iznad dubine aglomeracije, a kojeg izvodi glavni proces.

# procesa	Trajanje (ms)	Ubrzanje	Učinkovitost
Dubina alomeracije = 1 -> # zadataka = 7			
2	10820	0.9837	0.4919
3	6211	1.7137	0.5712
4	4342	2.4514	0.6129
5	3373	3.1556	0.6311
6	3184	3.3430	0.5572
Dubina alomeracije = 2 -> # zadataka = 49			
2	11192	0.9510	0.4755
3	5646	1.8852	0.6284
4	3907	2.7243	0.6811
5	3137	3.3931	0.6786
6	2750	3.8705	0.6451
Dubina alomeracije = 3 -> # zadataka = 343			
2	10871	0.9791	0.4896
3	5543	1.9203	0.6401
4	3759	2.8316	0.7079
5	2936	3.6253	0.7251
6	2428	4.3839	0.7306
Dubina alomeracije = 4 -> # zadataka = 2401			
2	11048	0.9634	0.4817
3	5691	1.8703	0.6234
4	3773	2.8211	0.7053
5	3014	3.5315	0.7063
6	2508	4.2440	0.7073



